

CHAPTER 14

Prove Every Token You Saved

Building Evidence-Grade Agent Economics That Can Survive an Audit

Experimental Specification v1.0

HarborPoint Distribution Group | Controlled Agent-Economics Attribution Benchmark

Purpose. Define the experiment before implementation and before any canonical real-provider run. This chapter does not ask whether a Smart Agent can reduce LLM use; earlier chapters already measured that. It asks whether the system can *prove* where every claimed saving came from, reconcile provider usage to request-level evidence, distinguish avoided reasoning from merely shifted cost, and prevent flattering but unsupported ROI claims.

Companion repository target:

<https://github.com/juancabreramware/smart-ai-agents/tree/main/chapter-14-prove-every-token-you-saved>

Frozen principle. A saving is not evidence because a dashboard says it happened. A saving is evidence only when the counterfactual, actual execution, token usage, pricing, deterministic overhead, failures, retries, relearning, and reconciliation rules are all explicit enough for an independent auditor to reproduce the claim.

1. Research Question

Can an agent-economics system produce request-level, independently auditable savings claims that reconcile to provider usage and remain correct when the workload contains deterministic reuse, fresh reasoning, retries, failures, relearning, pricing changes, and non-LLM execution costs?

The experiment is intentionally about **measurement quality**, not about manufacturing a large savings percentage. A result in which the evidence-grade system reports smaller savings than a naive dashboard is valid and potentially more important.

Primary questions

Question	What must be demonstrated
Q1 - Reconciliation	Can recorded LLM calls and tokens reconcile exactly, or within an explicitly frozen tolerance, to provider-returned usage for the benchmark run?
Q2 - Attribution	Can each avoided call be tied to a specific deterministic route and a defined baseline counterfactual?
Q3 - Full economics	Can the system separate measured provider cost from deterministic runtime, validation, storage/registry, maintenance, and relearning costs?
Q4 - Failure accounting	Do retries, failed model calls, invalidations, and relearning increase actual cost rather than disappearing from the savings calculation?
Q5 - Auditability	Can an independent script recompute all headline metrics from immutable ledgers without trusting summary.json?

2. What Is Being Compared

The benchmark uses one frozen workload and one frozen Smart execution policy, but evaluates three accounting architectures over the resulting execution evidence. This avoids confusing execution quality with measurement quality.

Accounting architecture	Behavior
A. Naive Token Dashboard	Counts visible successful LLM calls and their tokens. Treats deterministic routes as free. Ignores failed/retried calls, validation and relearning overhead, and uses a single current price table. Produces a plausible but incomplete savings claim.

Accounting architecture	Behavior
B. Aggregate Cost Reporter	Counts all provider-returned calls/tokens and uses frozen pricing by model, but aggregates at run level. Includes declared deterministic overhead but cannot prove request-level counterfactual attribution for every saving.
C. Evidence-Grade Economics Ledger	Records immutable request-level actual and counterfactual evidence, route reason, model usage, pricing version, retries/failures, deterministic overhead, validation, invalidation/relearning, and reconciliation fields. All headline metrics are recomputed by an independent auditor.

The Smart execution path itself must be identical for B and C. The benchmark must never give the evidence-grade ledger a better routing policy merely to make its economics look better.

3. Counterfactual Design

Savings require a baseline. The canonical counterfactual is a frozen **LLM-every-request baseline** processing the same ordered requests with the same public business contracts. The Smart path processes the same workload using deterministic reuse when compatible and the LLM when required.

The baseline is executed, not estimated, in the canonical benchmark. Therefore the primary savings claim compares measured baseline provider usage against measured Smart provider usage. No chapter headline may multiply Smart avoided-call count by an invented average token value.

Primary provider savings = measured baseline provider cost - measured Smart provider cost
 Full benchmark savings = baseline full cost - Smart full cost
 Smart full cost = measured provider cost + declared deterministic execution cost + declared validation cost + declared registry/storage cost + declared maintenance/relearning cost

Synthetic non-provider costs are permitted only when declared as **benchmark-assigned costs**, frozen before the canonical run, stored in a cost manifest, and never described as market-measured costs.

4. Frozen Workload Shape

The implementation must create exactly **240 requests** across four ordered waves of 60. Chapter 14 deliberately reuses the four-wave mental model from Chapter 13 so readers can focus on accounting rather than learn a new workload grammar. The exact request interleaving and values will be generated deterministically and hashed before the real run.

Wave	Purpose	Required accounting stress
Wave 1 - Learn	Initial acquisitions and recurring work	Fresh model calls, capability validation, first deterministic promotions.
Wave 2 - Reuse	Heavy compatible deterministic reuse	Large number of avoided calls; ledger must attribute each saving to an eligible baseline counterfactual.
Wave 3 - Drift + Failure	Controlled contract drift plus injected provider retry/failure scenarios	Invalidation, relearning, retries, failed calls, and changed cost must all remain visible.
Wave 4 - Mature + Mixed Models	Stable reuse plus novelty; optional second model/pricing epoch	Tests model-specific pricing, mature reuse, novelty, and end-to-end reconciliation.

The exact workload hash, failure schedule, model schedule, price epochs, and benchmark-assigned cost manifest are implementation-time frozen artifacts. They must be committed to the evidence manifest before the canonical run.

5. Required Event Types

The benchmark must contain enough event diversity to make weak accounting visibly wrong. At minimum, the canonical workload must exercise the following event classes.

Event	Accounting requirement
Fresh reasoning	Count actual provider usage and cost; tie result to request ID.
Deterministic reuse	Zero provider call for actual Smart path; preserve baseline counterfactual evidence and deterministic execution cost.
Reasoning-required exception	No savings may be claimed merely because a capability exists.
Capability acquisition	Record provider usage plus validation/acquisition overhead.
Contract drift	Invalidate before reuse; record invalidation and any relearning expense.
Provider retry	Every attempted provider call must be counted, including the failed attempt when usage/cost is returned.
Provider hard failure	Record failure outcome and any billable usage. Do not count the request as a successful saving.
Model/pricing epoch change	Apply the price table effective for that call, not one global end-of-run price.

6. Evidence-Grade Request Ledger

Every Smart and baseline request must have a stable request ID. The evidence-grade ledger is append-only during the run. Summary files are derived products, never the source of truth.

```
{ "request_id": "REQ-0147", "wave": 3, "family": "invoice", "actual_route": "relearning",
  "counterfactual_route": "llm_reason", "model": "gpt-5-mini", "pricing_epoch": "P2", "attempts": 2,
  "input_tokens": 418, "output_tokens": 171, "provider_cost_micro": 446, "deterministic_cost_micro": 0,
  "validation_cost_micro": 120, "relearning_cost_micro": 250, "correct": true, "stale_reuse_error": false,
  "capability_id": "invoice:v2", "evidence_hash": "..."} }
```

Fields shown above are illustrative schema requirements, not benchmark results. Exact numeric cost constants are frozen later in the cost manifest.

7. Pricing and Cost Provenance

Provider prices used for the experiment must be frozen into a versioned pricing manifest immediately before the canonical run. The manifest records model name, input price, output price, currency, effective benchmark epoch, and provenance. The chapter must report these as **benchmark pricing constants**, even when they match public provider pricing at run time.

Non-provider costs use integer microdollars to avoid floating-point drift. Each category must have a named basis and must be separately reportable so readers can see provider-only savings and full benchmark economics.

Cost bucket	Source	Headline treatment
LLM provider cost	Measured provider token usage × frozen model/epoch pricing	Measured
Deterministic execution	Frozen benchmark-assigned per execution cost	Assigned; shown separately
Validation/acquisition	Frozen benchmark-assigned event cost	Assigned; shown separately
Registry/storage	Frozen benchmark-assigned per operation or per run	Assigned; shown separately
Relearning/maintenance	Frozen benchmark-assigned event cost	Assigned; shown separately
Expected error cost	Excluded from primary headline unless a predeclared, defensible model is frozen	Optional sensitivity analysis only

8. Reconciliation Invariants

The canonical audit must fail if any of these invariants fail.

Invariant	Required check
I1 Request conservation	Exactly 240 unique workload requests per execution architecture; no missing or duplicate request IDs.
I2 Attempt conservation	Every provider attempt appears exactly once in the provider-attempt ledger.
I3 Token conservation	Run-level input/output totals equal the sum of provider attempts.
I4 Cost conservation	Provider cost recomputed from attempt-level usage and pricing epoch equals reported provider cost.
I5 Route conservation	Every Smart request has exactly one terminal route classification.
I6 Savings attribution	Every claimed avoided call maps to one Smart deterministic request and one eligible baseline request.
I7 No free failures	Retries, failures, validation, and relearning costs cannot be dropped from full-cost accounting.
I8 Summary independence	Deleting summary.json must not prevent the auditor from recomputing headline results.
I9 Manifest integrity	Workload, policy, pricing, cost, and public-contract hashes match the run manifest.
I10 No future leakage	Routing/accounting logic cannot access future workload counts, drift schedule, or hidden ground truth.

9. Primary Metrics

Metric	Definition
Correctness	Correct / total requests, reported for baseline and Smart execution.
Provider calls	All provider attempts plus successful-call count; retries reported separately.
Input/output/total tokens	Measured provider-returned usage.
Measured provider cost	Attempt-level token usage × frozen pricing epoch.
Deterministic reuses	Requests completed without an LLM provider call because a compatible validated capability applied.
Avoided calls	Eligible baseline calls minus Smart provider calls, with request-level attribution.
Gross provider savings	Baseline measured provider cost - Smart measured provider cost.
Full benchmark savings	Baseline full cost - Smart full cost after declared assigned overhead.
Savings overstatement	Naive dashboard claimed savings - evidence-grade recomputed savings.
Reconciliation error	Absolute and percentage difference between reported and independently recomputed totals.
Evidence coverage	Percent of headline savings dollars traceable to request-level counterfactual + actual records.

10. Hypotheses

ID	Pre-registered hypothesis
H1	The evidence-grade ledger will reconcile provider calls, tokens, and provider cost exactly from attempt-level evidence, subject only to explicitly documented provider rounding.
H2	The Naive Token Dashboard will overstate full benchmark savings when retries, validation, deterministic overhead, and relearning are present.
H3	Aggregate Cost Reporter will compute provider totals correctly but will have lower request-level savings attribution coverage than the evidence-grade ledger.
H4	Evidence-grade accounting will preserve 100% attribution coverage for every claimed avoided call.
H5	Wave 3 will visibly increase Smart cost because drift, retries, and relearning are accounted rather than hidden.

ID	Pre-registered hypothesis
H6	The evidence-grade savings percentage may be smaller than the naive percentage; that outcome is valid and must not be tuned away.

11. Experimental Gates

Gate	Must pass before proceeding
G1 - Unit tests	All accounting, pricing, hashing, retry, attribution, and audit tests pass.
G2 - Boundary review	OpenAI provider receives only current request/public contract; no hidden ground truth or future schedule.
G3 - Mock benchmark	240-request deterministic mock completes; all three accounting reports generated.
G4 - Mock audit	Independent auditor recomputes all metrics and intentionally detects seeded accounting defects in negative tests.
G5 - Real demo	Small real-provider demo verifies token usage fields, pricing calculation, retry ledger behavior, and reconciliation.
G6 - Freeze manifests	Workload, policy, public contracts, pricing, cost manifest, failure schedule, and audit version hashed.
G7 - Canonical real run	Exactly one intended canonical run on frozen artifacts.
G8 - Independent audit	Audit passes with errors=[] and recomputed metrics equal published metrics.
G9 - Freeze evidence	Evidence ZIP created only after audit pass; SHA-256 and size recorded.
G10 - Publication	Draft uses only frozen canonical evidence for empirical claims; assigned costs labeled as assigned.

12. Negative Tests the Auditor Must Catch

Chapter 14 is an audit chapter, so a passing happy-path test suite is insufficient. The implementation must include deliberate tamper/defect tests proving the auditor rejects bad evidence.

Injected defect	Expected audit result
Delete one provider attempt	FAIL - attempt/token/cost conservation.
Duplicate one request ID	FAIL - request conservation.
Change one pricing epoch	FAIL - cost recomputation mismatch.
Mark an LLM-routed request as avoided	FAIL - savings attribution mismatch.
Drop a retry from full cost	FAIL - provider-attempt reconciliation.
Modify workload after run	FAIL - workload hash mismatch.
Edit summary.json only	FAIL - summary mismatch while ledger recomputation remains authoritative.
Insert future drift field into routing evidence	FAIL - anti-leakage rule.

13. Anti-Gaming Rules

The following are prohibited after canonical results are visible: changing workload composition to increase reuse; removing retry/failure events; changing assigned overhead to improve ROI; changing the baseline counterfactual; excluding expensive relearning; switching pricing constants; redefining an avoided call; or rerunning solely because the measured savings are smaller than expected.

If a methodological defect is discovered after the canonical run, preserve the original evidence, document the defect, version the specification, and run a new benchmark only when the defect materially invalidates the claim. Never silently replace an inconvenient result.

14. Evidence Package

The frozen evidence ZIP must contain at least:

```
manifest.json workload.jsonl ground_truth.jsonl baseline_request_ledger.jsonl smart_request_ledger.jsonl
provider_attempts.jsonl capability_events.jsonl pricing_manifest.json cost_manifest.json
failure_schedule.json naive_dashboard.json aggregate_report.json evidence_grade_report.json wave_summary.json
audit_report.json README.md
```

The evidence package must be sufficient for an independent reader to recompute the published totals without network access and without trusting generated summaries.

15. Planned Figures for Chapter 14

Figure	Purpose
14.1 - From Claim to Proof	Request → actual route → provider attempts → cost ledger → counterfactual → auditable savings.
14.2 - Three Accounting Architectures	Naive dashboard vs aggregate reporter vs evidence-grade ledger.
14.3 - Where Savings Disappear	Waterfall from gross provider savings through validation, deterministic execution, retries, relearning, and maintenance to full benchmark savings.
14.4 - Reconciliation Dashboard	Baseline vs Smart calls/tokens/cost plus evidence coverage and reconciliation error.
14.5 - Wave Economics	Per-wave provider and full benchmark cost showing the visible drift/retry bump.
14.6 - Audit Failure Matrix	Examples of tampering or omissions and the invariant that catches each.

16. Publication Rules

The chapter must distinguish four kinds of numbers: **measured provider usage**, **derived arithmetic**, **benchmark-assigned non-provider costs**, and **interpretation**. These categories must not be blended.

No benchmark number belongs in the manuscript until the canonical evidence has passed the independent audit and the evidence ZIP is frozen. Mock and demo results may be discussed as validation steps but must not be presented as the chapter's empirical result.

If the naive dashboard overstates savings dramatically, the chapter should show that result. If it does not, the chapter should show that too. The purpose is to prove savings, not to maximize the gap between accounting methods.

17. Implementation Constraints

Constraint	Requirement
Runtime	Python 3.13.1 target; commands use python, never py.
Provider	Real-provider path supports GPT-5-mini; structured outputs preferred.
Safety	No eval, exec, arbitrary shell, arbitrary SQL, or arbitrary network execution.
Arithmetic	Integer microdollars for benchmark accounting; deterministic rounding rules.
Files	Append-only JSONL ledgers during execution; derived JSON summaries afterward.
Reproducibility	Deterministic workload generator with frozen seed and SHA-256.
Audit independence	Auditor reads ledgers/manifests and recomputes; it does not import trusted summary calculations from runner.
Canonical protection	Environment confirmation required before real canonical run; freeze script refuses before audit pass.

18. Success Criteria

Chapter 14 succeeds if the final experiment can answer, with immutable evidence: exactly how many model calls occurred; exactly how many tokens were consumed; which price applied to each call; what deterministic and validation overhead was assigned; which requests avoided model reasoning; which baseline requests justify those avoided-call

claims; what retries, failures, invalidations, and relearning cost; and whether an independent auditor can reproduce the headline savings from the raw evidence.

The experiment does not require the Smart Agent to produce a predetermined savings percentage. A smaller but fully auditable saving is stronger evidence than a larger number that cannot be reconciled.

19. Frozen Principle

Do not optimize the benchmark to make the savings look impressive. Optimize the evidence so that every claimed saving can be traced, recomputed, challenged, and defended.

Any material semantic change to workload design, counterfactual definition, accounting rules, pricing treatment, cost categories, failure schedule, audit invariants, or canonical execution policy requires a versioned specification amendment before the canonical run.